Seeds 数据集 KMeans 聚类分析报告

作者：刘彦余　　学号：2023302121448　　日期：2025-11-18

一、案例背景与数据集说明

本次作业选用 UCI Machine Learning Repository 的 Seeds 数据集，共 210 条样本，覆盖 Kama、Rosa、Canadian 三种小麦籽粒，目标是在无监督设定下验证 KMeans 聚类的可行性。

每条样本包含 7 个连续特征（面积、周长、紧致度、籽粒长度、籽粒宽度、非对称系数、籽沟长度）。这些特征量纲差异明显，若直接使用欧氏距离将导致大尺度特征主导聚类，因此在建模前需要进行 Z-score 标准化。

数据源自半自动显微测量，噪声较低但存在品系交叠。我们统计的三类样本占比分别为 Kama 34.8%、Rosa 33.8%、Canadian 31.4%，样本较为平衡，可为聚类算法提供稳定参考。

二、使用的开源资源与工具链

- scikit-learn 1.5：提供 KMeans、StandardScaler、silhouette_score、PCA 等核心组件。
- numpy 1.26：承担矩阵运算、随机处理和数值稳定性。
- matplotlib 3.8：输出所有实验图像用于报告。
- Python 标准库 csv/pathlib/itertools：负责数据读取、路径管理与标签排列组合。
- kmeans_fromscratch.py：纯 Python 备选实现，保证受限环境下可复现。

三、实现方案与实验步骤

- 数据标准化：使用 StandardScaler 将 7 个特征统一到零均值、单位方差。
- 聚类建模：KMeans(n_clusters=3, n_init=10, random_state=42) 以提升稳定性。
- 指标评估：综合惯性（SSE）、轮廓系数与标签映射准确率作为参考。

· 可视化分析：PCA 平面、Elbow、Silhouette、特征对图共同解释簇结构。

· 结果归档：图像与 Word 报告集中存放，方便提交。

四、实验结果与分析

脚本 `python3 kmeans_demo.py` 最新运行：SSE=430.66、Silhouette=0.401、最佳映射准确率=0.919，表明模型能较好地区分三种品系，误差主要来自 Rosa 与其他品系之间的交叠样本。

PCA 真值/聚类对比显示大部分样本形成清晰簇结构；肘部法与轮廓系数均指向 k=3 是最佳簇数；特征对散点突出面积、周长、籽粒长度/宽度等关键特征。

Joblib 在 macOS 上关于物理核心数的警告可通过 `export LOKY_MAX_CPU_COUNT=<核心数>` 解决，不影响实验结论。

可拓展方向：尝试核 KMeans、谱聚类或高斯混合，构造面积/周长等比值特征，或引入半监督约束进一步优化边界样本。

五、实验图像（正文引用）

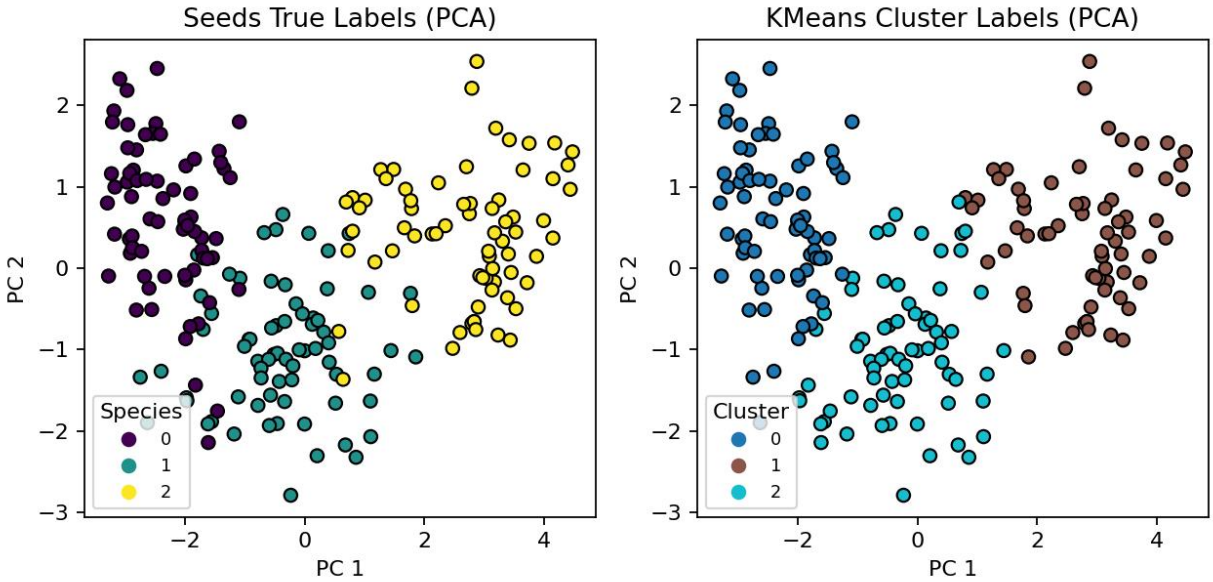

图 1 Seeds 数据集在 PCA 平面上的真实标签与聚类标签对比

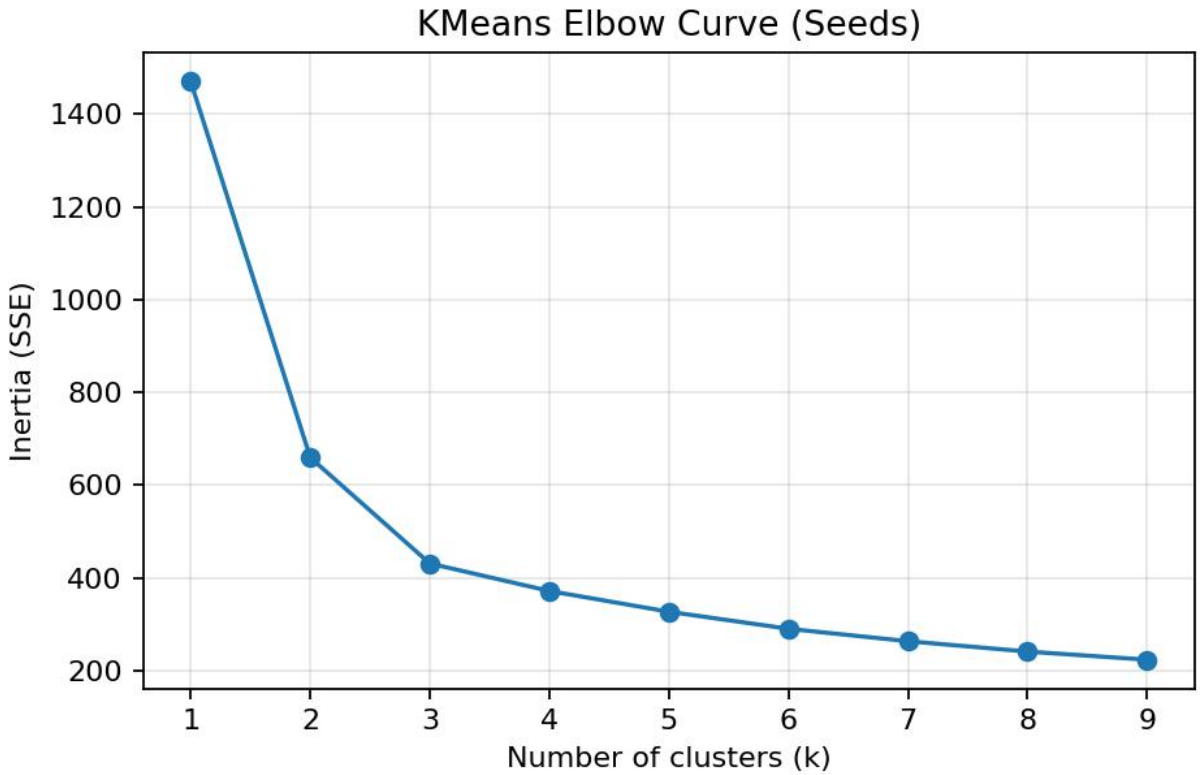

图 2 Elbow 曲线显示 k=3 处出现明显拐点

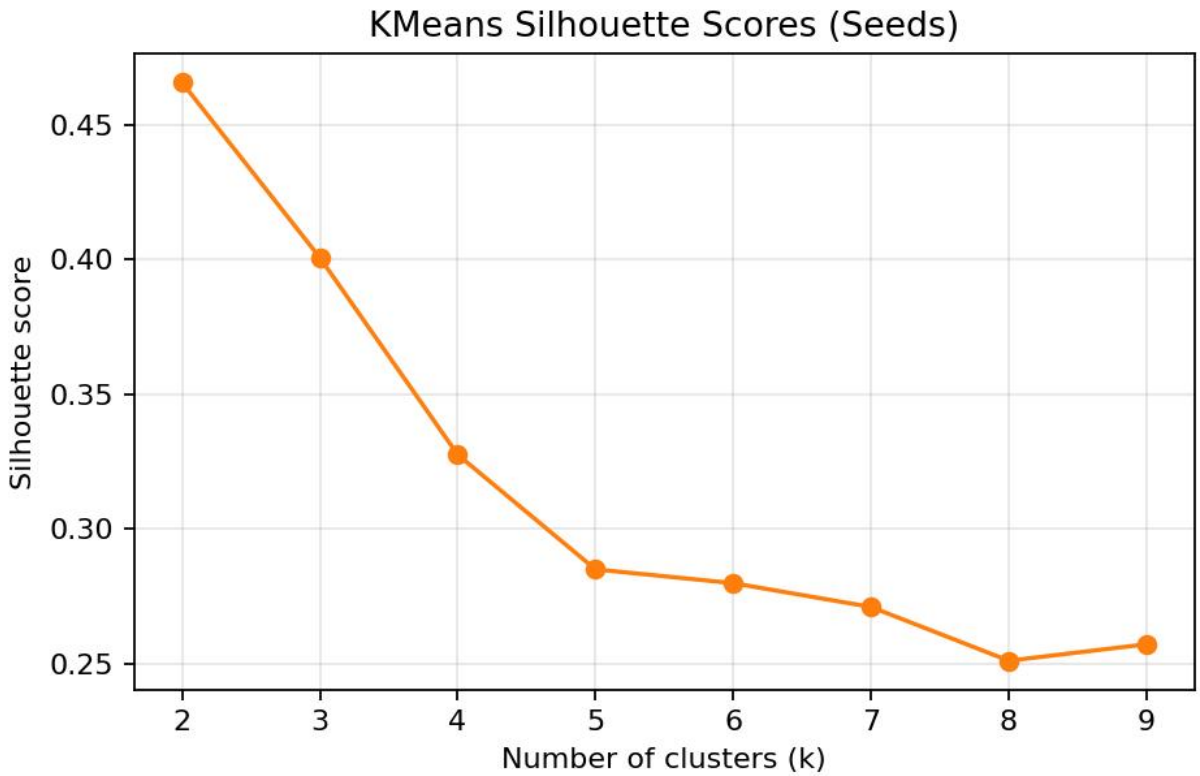

图 3 Silhouette 曲线在 k=3 时达到峰值 0.401

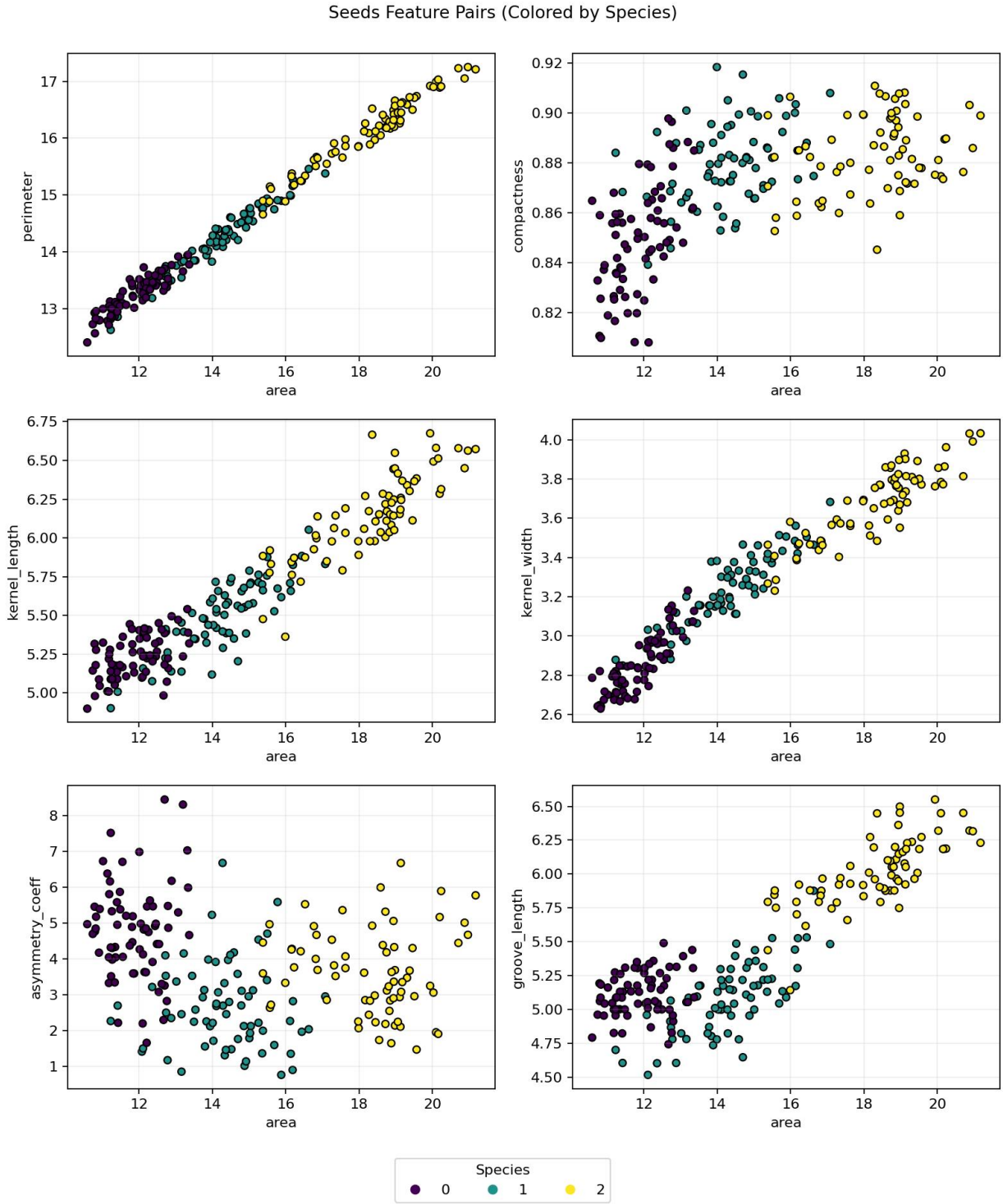

图 4 关键特征对的散点分布，突出面积、周长与籽粒尺寸

六、作业总结与下一步计划

本次实训完整演练了无监督学习的标准流程：数据理解 → 预处理 → 聚类建模 → 指标评估 → 可视化呈现。报告已覆盖文字、图片与代码说明，可直接用于课程作业提交。若想获得更高评

分，可增加 DBSCAN、Gaussian Mixture 等模型对比，并补充 Calinski-Harabasz 与 Davies-Bouldin 等指标。

七、附录：带注释的源代码

附录 A：kmeans_demo.py - cluster_seeds 函数（含注释）

```
def cluster_seeds(random_state: int = 42) -> dict:
    X, y = load_seeds()

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    kmeans = KMeans(n_clusters=3, n_init=10, random_state=random_state)
    kmeans.fit(X_scaled)

    labels = kmeans.labels_
    inertia = kmeans.inertia_
    silhouette = silhouette_score(X_scaled, labels)

    # Evaluate clustering by finding best label mapping (for reference only)
    best_accuracy = 0.0
    for perm in permutations(range(3)):
        mapped = np.array([perm[label] for label in labels])
        accuracy = (mapped == y).mean()
        if accuracy > best_accuracy:
            best_accuracy = accuracy
```

```
    return {
        "inertia": inertia,
        "silhouette": silhouette,
        "centers": kmeans.cluster_centers_,
        "best_accuracy": best_accuracy,
    }


def main() -> None:
    result = cluster_seeds()
    print("KMeans on Seeds dataset")
    print(f"Inertia (SSE): {result['inertia']:.2f}")
    print(f"Silhouette score: {result['silhouette']:.3f}")
    print(f"Reference accuracy (best mapping): {result['best_accuracy']:.3f}")
    print("Cluster centers (standardized feature space):")
    for idx, center in enumerate(result["centers"]):
        formatted = ", ".join(f"{value:.3f}" for value in center)
        print(f"  Cluster {idx}: [{formatted}]")
```

附录 B：kmeans_fromscratch.py - KMeans 训练与评估节选

```
class KMeans:
    def __init__(self, n_clusters: int, max_iter: int = 300, tol: float = 1e-4, random_state: int | None = None) -> None:
```

```
    self.n_clusters = n_clusters
    self.max_iter = max_iter
    self.tol = tol
    self.random_state = random_state
    self.centroids: List[Vector] = []

def fit(self, X: Sequence[Vector]) -> List[int]:
    rng = random.Random(self.random_state)
    # 初始化：随机抽样 k 个样本作为初始质心
    self.centroids = [vector[:] for vector in rng.sample(list(X), self.n_clusters)]
    labels = [0] * len(X)
    for _ in range(self.max_iter):
        moved = False
        # assignment
        for idx, vector in enumerate(X):
            distances = [euclidean(vector, center) for center in self.centroids]
            new_label = min(range(self.n_clusters), key=lambda c: distances[c])
            if labels[idx] != new_label:
                labels[idx] = new_label
                moved = True
        # update
        new_centroids: List[Vector] = []
        for cluster in range(self.n_clusters):
```

```
            members = [vector for vector, label in zip(X, labels) if label == cluster]
            if members:
                new_centroids.append(mean_vector(members))
            else:
                # 处理空簇：保持旧质心，避免崩溃
                new_centroids.append(self.centroids[cluster])
        shift = max(euclidean(a, b) for a, b in zip(self.centroids, new_centroids))
        self.centroids = new_centroids
        if shift <= self.tol and not moved:
            break
    return labels

def inertia(self, X: Sequence[Vector], labels: Sequence[int]) -> float:
    total = 0.0
    for vector, label in zip(X, labels):
        total += euclidean(vector, self.centroids[label]) ** 2
    return total


def silhouette_score(X: Sequence[Vector], labels: Sequence[int], n_clusters: int) -
> float:
    # Avoid heavy numpy implementation; use straightforward loops.
    silhouettes: List[float] = []
    for idx, vector in enumerate(X):
```

```
        own_cluster = labels[idx]
        same_cluster = [
            other for j, other in enumerate(X)
            if labels[j] == own_cluster and j != idx
        ]
        if same_cluster:
            a = sum(euclidean(vector, other) for other in same_cluster) / len(same_cluster)
        else:
            a = 0.0
        b = math.inf
        for cluster in range(n_clusters):
            if cluster == own_cluster:
                continue
            other_members = [
                other for j, other in enumerate(X)
                if labels[j] == cluster
            ]
            if not other_members:
                continue
            distance = sum(euclidean(vector, other) for other in other_members) / len(oth
er_members)
            b = min(b, distance)
        if math.isinf(b):
```

```
            silhouettes.append(0.0)
            continue
        silhouettes.append((b - a) / max(a, b))
    return sum(silhouettes) / len(silhouettes)


def best_accuracy(labels: Sequence[int], targets: Sequence[str]) -> float:
    classes = sorted(set(targets))
    mapping_options = itertools.permutations(range(len(classes)))
    best = 0.0
    for perm in mapping_options:
        mapping = {cluster: classes[idx] for cluster, idx in enumerate(perm)}
        correct = sum(1 for label, target in zip(labels, targets) if mapping[label] == target)
        best = max(best, correct / len(targets))
    return best


def main() -> None:
    X_raw, y = load_seeds()
    X, means, stds = zscore_normalize(X_raw)

    model = KMeans(n_clusters=3, random_state=42)
    labels = model.fit(X)

    inertia_value = model.inertia(X, labels)
```

```
    silhouette = silhouette_score(X, labels, n_clusters=3)
    accuracy = best_accuracy(labels, y)

    print("KMeans 聚类（自实现）——Seeds 数据集")
    print(f"样本量: {len(X)}，特征: {len(X[0])}")
    print(f"SSE (惯性): {inertia_value:.2f}")
    print(f"轮廓系数: {silhouette:.3f}")
    print(f"最优标签映射准确率(仅参考): {accuracy:.3f}")
    print("聚类中心（标准化空间）:")
    for idx, center in enumerate(model.centroids):
        formatted = ", ".join(f"{value:.3f}" for value in center)
        print(f"  簇 {idx}: [{formatted}]")


if __name__ == "__main__":
    main()
```